MODULE 08 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Choosing a workload

StatefulSets, DaemonSets, Jobs and CronJobs, and how to pick the right one in seconds.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 8 of 35

admin@sthithapragna.com

Ask one question: are the Pods identical, individual, everywhere, or finite?

The workload decision table

IF THE PODS...	USE	QUAYSIDE EXAMPLE
are interchangeable copies	Deployment	Web front end, booking API
each need a stable name and their own storage	StatefulSet	Bookings database
must run once on every node	DaemonSet	Log agent
run once to completion	Job	A one-off fare recalculation
run to completion on a schedule	CronJob	Nightly manifest export

ON THE EXAM

Most exam scenarios contain one phrase that points straight at a row of this table. Find it.

Deployment versus StatefulSet

DEPLOYMENT

Cattle

- Random names: web-7d9f6c-x2k4p
- Pods are interchangeable
- Shared or no storage
- Started and stopped in any order

STATEFULSET

Named individuals

- Ordinal names: bookings-db-0, -1, -2
- Each Pod keeps its identity
- Its own volume per Pod
- Ordered start-up and shut-down

THE TIEBREAKER

Both run Pods from one identical template. The difference is identity and storage, not the container.

What a StatefulSet guarantees

- **A stable name.** bookings-db-0 is always bookings-db-0, even after being rescheduled.
- **A stable DNS name** through a headless Service: bookings-db-0.bookings-db.quayside.svc.cluster.local.
- **Stable storage.** Each Pod's own claim, such as data-bookings-db-0, is reattached when it is rescheduled.
- **Order.** Created 0, 1, 2 by default; removed in reverse; updated from the highest ordinal down.

> ON THE EXAM
>
> Identity protects against split brain: two Pods can never both believe they are member 0.

Quayside's bookings database as a StatefulSet

Trimmed. Selector and Pod template are as for a Deployment.

```
 1 apiVersion: apps/v1
 2 kind: StatefulSet
 3 metadata: {name: bookings-db}
 4 spec:
 5   serviceName: bookings-db
 6   replicas: 3
 7   # selector and Pod template as usual;
 8   # the container mounts volume "data"
 9   volumeClaimTemplates:
10   - metadata: {name: data}
11     spec:
12       accessModes: [ReadWriteOnce]
13       resources: {requests: {storage: 50Gi}}
```

5 The headless Service that gives each Pod its DNS name.

9-10 A template for claims: one volume per Pod, named data-bookings-db-0 and so on.

Headless Services, and what StatefulSets do not do

- **Headless Service:** clusterIP: None. DNS returns each Pod's own IP instead of one virtual IP.
- **No clustering magic.** Replication and failover are the database's job, or an operator's.
- **Claims are kept by default** when you scale down or delete. The retention policy can change that.
- **For real databases,** an operator built on a StatefulSet is often the better answer.

> ON THE EXAM
>
> A StatefulSet gives identity and storage. Making Postgres replicate is still Postgres's job.

DaemonSets: one Pod on every node

- **One copy per node,** or per selected node using nodeSelector or affinity.
- **New nodes get a Pod automatically;** removed nodes' Pods are cleaned up.
- **Typical uses:** log collectors, node monitoring agents, network plugins, storage drivers.
- kube-proxy itself usually runs as a DaemonSet. Tolerations let it reach control plane nodes too.

> ON THE EXAM
>
> Anything that must exist on every node, including nodes that do not exist yet, is a DaemonSet.

DaemonSet versus a Deployment sized to nodes

DEPLOYMENT, REPLICAS = NODES

Looks equivalent

- Scheduler may stack two on one node
- Leaves some nodes with none
- Ignores nodes added later
- Needs manual resizing

DAEMONSET

Is equivalent

- Exactly one per eligible node
- Every node covered
- Follows nodes as they come and go
- No count to maintain

THE TIEBREAKER

'On every node, including new ones' is always a DaemonSet.

Jobs: work that finishes

- **Runs Pods to completion** and records success. restartPolicy must be OnFailure or Never.
- **completions** is how many successes are needed; **parallelism** is how many run at once.
- **backoffLimit**, 6 by default, is how many failures before the Job gives up.
- **activeDeadlineSeconds** caps total run time; **ttlSecondsAfterFinished** cleans it up afterwards.

> ON THE EXAM
>
> A Deployment restarts a finished program forever. A Job knows when to stop.

Job shapes

SHAPE	SETTINGS	GOOD FOR
One-off	completions 1, parallelism 1	A single task, such as a migration
Fixed count, in parallel	completions 20, parallelism 5	20 pieces of work, 5 at a time
Indexed	completionMode: Indexed	Each Pod gets its own index, and its own slice of work
Work queue	parallelism N, completions unset	Workers pull from a queue until it is empty

ON THE EXAM

Split a big batch into numbered chunks: an Indexed Job with completions and parallelism.

What happens when a CronJob fires

01
Schedule time
The CronJob's schedule matches the clock.

02
CronJob controller
Creates a Job from the jobTemplate.

03
Job controller
Creates a Pod from the Job's template.

04
Scheduler, kubelet
Place the Pod and run it to completion.

05
History
Keeps 3 successful and 1 failed Job by default.

ON THE EXAM

CronJob creates Jobs. Jobs create Pods. The kubelet never reads a CronJob.

Quayside's nightly manifest export

```
1  apiVersion: batch/v1
2  kind: CronJob
3  metadata: {name: manifest-export}
4  spec:
5    schedule: "30 2 * * *"
6    timeZone: Europe/London
7    concurrencyPolicy: Forbid
8    jobTemplate:
9      spec:
10       backoffLimit: 3
11       template:
12         spec:
13           restartPolicy: OnFailure
14           containers: [{name: export, image: quayside/export:1.8.0}]
```

5-6 02:30 every day, London time rather than the controller's clock.

7 If last night's run is still going, skip tonight's.

13 Retry the container in place on failure.

Reading a cron schedule

Five fields: minute, hour, day of month, month, day of week.

SCHEDULE	RUNS
`"0 * * * *"`	At the start of every hour
`"30 2 * * *"`	At 02:30 every day
`"0 9 * * 1"`	At 09:00 every Monday
`"*/15 * * * *"`	Every 15 minutes
`"@daily"`	A shortcut for midnight every day

ON THE EXAM

Read left to right: minute first, then hour. "30 2" is 02:30, not 2:30 in the afternoon.

Controlling overlapping runs

SETTING	EFFECT
`concurrencyPolicy: Allow`	Default. A new run starts even if the last is still going.
`concurrencyPolicy: Forbid`	Skip the new run while the last is still going.
`concurrencyPolicy: Replace`	Stop the running Job and start the new one.
`startingDeadlineSeconds`	Skip a run that could not start within this many seconds.
`suspend: true`	Pause future runs. Jobs already running carry on.

ON THE EXAM

Allow is the default, and the cause of most 'why is this running twice' incidents.

Eight decisions. In each one, find the phrase that picks the workload.

> Quayside's manifest export must run at 02:30 London time every night, finish, and not run again until the next night.

Which workload should run the export?

A A Deployment whose container sleeps until 02:30

B A CronJob with a schedule and a time zone

C A Job, started by an engineer each night

D A DaemonSet so the export runs on every node

◆ PAUSE AND DECIDE

Which workload should run the export?

A A Deployment whose container sleeps until 02:30

Runs forever, and restarts the program after each finish.

B A CronJob with a schedule and a time zone

Creates a Job on schedule; the Job runs to completion.

C A Job, started by an engineer each night

Works, but depends on a person every single night.

D A DaemonSet so the export runs on every node

Would run the export once per node: many copies.

THE SIGNAL

'Finishes, then runs again on a schedule' is a CronJob.

> Quayside's log agent reads container logs from each node's disk and ships them to storage.
>
> It must run on every node, including nodes the autoscaler adds during the evening rush.

Which workload fits the log agent?

A. A Deployment with replicas equal to the node count

B. A StatefulSet so each agent has a stable identity

C. A CronJob that runs the agent every minute

D. A DaemonSet, one Pod on each eligible node

◆ PAUSE AND DECIDE

Which workload fits the log agent?

A A Deployment with replicas equal to the node count

Can stack copies on one node, and ignores new nodes.

B A StatefulSet so each agent has a stable identity

Identity is irrelevant; coverage of nodes is the need.

C A CronJob that runs the agent every minute

Would miss logs between runs, and still not cover every node.

D A DaemonSet, one Pod on each eligible node

One per node, and new nodes get a Pod automatically.

THE SIGNAL

'Every node, including new ones' is a DaemonSet.

> Quayside runs Postgres as three replicas. Each replica keeps its own data on disk, and the replicas find each other by name to replicate.
>
> If a replica is rescheduled, it must come back with the same name and the same data.

Which approach fits the database?

A. A StatefulSet, volumeClaimTemplates, headless Service

B. A Deployment of three replicas sharing one volume claim

C. A DaemonSet pinned to three labelled nodes

D. A Deployment of three replicas using emptyDir volumes

◆ PAUSE AND DECIDE

Which approach fits the database?

A A StatefulSet, volumeClaimTemplates, headless Service

Stable names, stable DNS and a volume per replica.

B A Deployment of three replicas sharing one volume claim

Random names, and three databases writing one volume.

C A DaemonSet pinned to three labelled nodes

Ties the database to nodes; no per-replica identity or claims.

D A Deployment of three replicas using emptyDir volumes

emptyDir is deleted with the Pod: data lost on reschedule.

THE SIGNAL

'Same name and same data after rescheduling' is a StatefulSet.

> After a fare change, Quayside must recalculate every stored fare once. The task takes about twenty minutes.
>
> If it fails, it should be retried. When it succeeds, it must not run again.

Which workload should run the recalculation?

A. A CronJob scheduled for a date far in the future

B. A Deployment of one replica that exits when done

C. A Job with a sensible backoffLimit

D. A bare Pod created with kubectl run

◆ PAUSE AND DECIDE

Which workload should run the recalculation?

A A CronJob scheduled for a date far in the future

A schedule for a one-off task is a workaround, not a design.

B A Deployment of one replica that exits when done

A Deployment restarts it after success, forever.

C A Job with a sensible backoffLimit

Runs once to completion and retries failures.

D A bare Pod created with kubectl run

No retry if its node fails, and no record of success.

THE SIGNAL

'Once, retry on failure, never again after success' is a Job.

> An engineer asks what actually happens inside the cluster when the manifest-export CronJob's scheduled time arrives.

Which description is correct?

A The kubelet watches CronJobs and starts the Pod itself

B The scheduler watches CronJobs and runs them on time

C The CronJob controller creates a Pod and waits for it

D The CronJob controller makes a Job; the Job makes a Pod

◆ PAUSE AND DECIDE

Which description is correct?

A The kubelet watches CronJobs and starts the Pod itself

The kubelet only runs Pods bound to its node.

B The scheduler watches CronJobs and runs them on time

The scheduler places Pods; it knows nothing about schedules.

C The CronJob controller creates a Pod and waits for it

It skips a step: CronJobs never create Pods directly.

D The CronJob controller makes a Job; the Job makes a Pod

CronJob, then Job, then Pod, each by its own controller.

THE SIGNAL

CronJob creates Job; Job creates Pod. Each object has its own controller.

> A reconciliation CronJob runs every 15 minutes. Usually it takes 5 minutes, but at month end it can take 25.
>
> Two runs at once corrupt the same report file. Finishing each run matters more than starting on time.

Which setting prevents the problem?

A concurrencyPolicy: Forbid

B concurrencyPolicy: Replace

C concurrencyPolicy: Allow

D startingDeadlineSeconds: 60

◆ PAUSE AND DECIDE

Which setting prevents the problem?

A concurrencyPolicy: Forbid
Skips a run while the previous one is still going.

B concurrencyPolicy: Replace
Kills the running job, which must be allowed to finish.

C concurrencyPolicy: Allow
The default, which is exactly what causes the overlap.

D startingDeadlineSeconds: 60
Limits late starts; does nothing about overlap.

THE SIGNAL

'Never overlap, and let the current run finish' is Forbid.

> Quayside's ticket validator is a stateless Go service. Every copy is identical and keeps no local data.
>
> It should scale from 4 to 20 copies with demand, as quickly as possible.

Which workload fits the validator?

A. A StatefulSet, so each copy has a stable name

B. A DaemonSet, so copies follow the nodes

C. A Deployment, scaled by demand

D. A Job with parallelism set to 20

◆ PAUSE AND DECIDE

Which workload fits the validator?

A. A StatefulSet, so each copy has a stable name
Adds ordering that slows scaling, for identity it does not need.

B. A DaemonSet, so copies follow the nodes
Ties the copy count to node count, not to demand.

C. A Deployment, scaled by demand
Interchangeable copies that scale freely.

D. A Job with parallelism set to 20
Jobs finish; the validator must keep serving.

THE SIGNAL

'Identical, stateless, scale freely' is a Deployment.

> Quayside must process one million archived tickets, already split into 20 numbered chunks. Each Pod should handle one chunk.
>
> The database can cope with at most 5 workers at a time.

How should this be run?

A. A CronJob that fires every minute, 20 times

B. An Indexed Job: completions 20, parallelism 5

C. A Deployment of 5 replicas that share the chunks

D. 20 bare Pods, created by a shell loop

◆ PAUSE AND DECIDE

How should this be run?

A. A CronJob that fires every minute, 20 times
Scheduling is the wrong tool; runs could overlap or be missed.

B. An Indexed Job: completions 20, parallelism 5
Each Pod gets a chunk index; five run at once.

C. A Deployment of 5 replicas that share the chunks
Deployments never finish, and nothing assigns chunks.

D. 20 bare Pods, created by a shell loop
No retries, no limit of five, no completion record.

THE SIGNAL

'Numbered chunks, a cap on concurrency' is an Indexed Job.

Module 8 in five lines

- **Identical**: Deployment. **Individual**: StatefulSet. **Everywhere**: DaemonSet. **Finite**: Job or CronJob.
- StatefulSets give **stable names, DNS via a headless Service, and a volume per Pod**.
- A DaemonSet **follows the nodes**; a Deployment sized to the node count does not.
- **CronJob creates a Job; the Job creates a Pod.** Default concurrency is Allow.
- Jobs: **completions, parallelism, backoffLimit** (6), Indexed for numbered chunks.

Every one of these finds its Pods by label. Next: **labels, namespaces, ConfigMaps and Secrets**.